A Topological Graph Neural Network Framework for Hardware-Agnostic Quantum Circuit Routing on NISQ Architectures

Jeriel Philly Koshy
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
jeriel.phillykoshy2024@vitstudent.ac.in

Nikhil Samson Sureshkumar
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
nikhil.samson2024@vitstudent.ac.in

Jennica Hanlynn Suhan
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
jennica.hanlynn2024@vitstudent.ac.in

Muthunagai S U
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
muthunagai.su@vit.ac.in

***Abstract*— In the field of Noisy Intermediate-Scale Quantum (NISQ) computing, when it is necessary to run arbitrary multi-qubit circuits on actual hardware, routing of non-adjacent logical two-qubit operations onto the physically constrained coupling topologies must be carried out. Because of these physical limitations, SWAP gates have to be inserted dynamically, which greatly increases the depth of the circuit, leads to gate errors, and speeds up quantum decoherence. Current compilers mainly depend on greedy heuristic search methods (for example, variants of SABRE) that do not have multi-step foresight, or on tabular machine learning and model-free reinforcement learning (RL) approaches that are unable to generalize to different chip topologies since they rely on rigid spatial memorization. To overcome these shortcomings, we introduce a Topological Graph Neural Network (GNN) Routing Framework for qubit mapping that is independent of specific hardware. This proposed architecture models both the physical coupling graph and the pending gate operations through spatial message passing. Each physical qubit is described by a 9-dimensional topological feature vector that includes the immediate gate operands, the shortest-path distances, and the forward-looking gate pressure over a fixed lookahead window. A 3-layer Graph Convolutional Network (GCN) with global mean pooling is used to predict the residual routing costs for possible SWAP operations so as to select paths with the lowest overhead. We thoroughly compare the GNN framework with classical greedy heuristics, tabular ensemble learning (Random Forest), and model-free Deep RL (PPO). Evaluated across a stress test of 1,000 deeply entangled random circuits (21–40 gates), the proposed GNN router reduces cumulative SWAP insertion from 30,104 to 23,431—achieving a 22.17% reduction in SWAP overhead over standard baseline routing demonstrating zero-shot transfer across the evaluated linear, branching, cyclic, and 7-qubit tree architectures without retraining: a 5-qubit T-junction (16.34% reduction), a 5-qubit cyclic ring (24.25% reduction), and an unseen 7-qubit IBM Falcon subgraph (15.61% reduction).**

Index Terms—Qubit Mapping, Quantum Circuit Compilation, Graph Neural Networks (GNN), NISQ Computing, SWAP Optimization, Reinforcement Learning, Lookahead Routing.

I. INTRODUCTION

Quantum computing has the potential to provide a tremendous increase in computational power in the fields of chemistry simulation, cryptography, and combinatorial optimization [1]. However, current implementations fall within the Noisy Intermediate-Scale Quantum (NISQ) period [2], involving processors having between tens and hundreds of qubits and lacking fault-tolerant quantum error correction. These NISQ machines are affected by short coherence times (T_1, T_2) and gate infidelities, especially in the case of two-qubit operations, where error rates (0.1%–1.0%) are over an order of magnitude higher than those of single-qubit gates [3]. It is therefore necessary to reduce both the depth of the circuit and the number of multi-qubit gates in order to maintain the fidelity of the quantum state before it is completely lost to decoherence.

In superconducting quantum processors (for example, IBM, Rigetti, and Google) there is a major architectural limitation in the form of restricted qubit connectivity [4]. The physical qubits are located on planar chips that are defined by a fixed coupling graph G=(V,E), and two-qubit operations (such as CNOT) can only take place between vertices that are directly adjacent (u,v)∈E. Although arbitrary quantum algorithms specify interactions between distant logical qubits, the compilers have to deal with the NP-complete Qubit Mapping and Routing Problem [5]: this involves permuting the virtual qubits along the physical paths by means of intermediate SWAP operations so that pairs of qubits which need to interact end up being adjacent before the

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

gate is carried out. Since one SWAP operation breaks down into three physical CNOT gates, the insertion of too many SWAP operations increases gate noise and quickly uses up the coherence budget [6].

In the past, quantum compilers have used classical combinatorial heuristics, for example greedy A* search and SABRE (SWAP-based Bounded-depth Routing) [7]. Although these heuristics are computationally efficient and adaptable to the topology, they depend on localized lookahead effects and often end up in local topological minima when applied to deep circuits.

Recent studies have looked into alternative methods using machine learning (ML) and reinforcement learning (RL) [8]–[11]. Tabular models (for example, Random Forests) estimate routing costs based on flattened state descriptions. Yet, the feature spaces used in tabular models associate qubit identities with fixed array indices, which leads to total failure whenever the models are transferred to reconfigured topologies or to chips that have defective qubits. On the other hand, Deep Reinforcement Learning (DRL) approaches (such as PPO and DQN) [12] suffer from serious bottlenecks in scaling the action space and experience policy collapse in the case of deep circuits, causing the agents to end up in cyclic SWAP loops.

In order to deal with these challenges, the present paper proposes a Topological Graph Neural Network (GNN) Routing Framework for hardware-agnostic compilation. Our approach represents both the chip topology and the sequences of gates that have not yet been executed as dynamic graph structures, and then uses spatial Graph Convolutional Networks (GCN) [13] to pass structural connectivity features directly between the hardware nodes. Instead of remembering the physical indices, the GNN learns the relational invariants associated with circuit interaction.

The primary contributions of this paper are:

1. **Cross-Topology GNN Router:** A 3-layer GCN architecture with global mean pooling that predicts residual routing overhead directly over coupling graphs, demonstrating empirical zero-shot transfer across linear, branching, cyclic, and 7-qubit tree architectures without retraining.
2. **Topological Feature Formulation:** A 9-dimensional representation of the node which combines the instantaneous gate inputs, the shortest-path graph geodesics, and a forward-looking "gate-pressure" metric over a bounded lookahead window.
3. **Multi-Paradigm Comparative Study:** A comparative study involving multiple paradigms has been carried out as an empirical benchmark, examining Greedy Heuristics, Tabular Supervised Learning (Random Forest), Deep RL (PPO), and the proposed GNN across 1,000 deep circuits (having between 21 and 40 gates).
4. **Empirical Validation:** Demonstration of a **22.17% reduction** on 1,000-circuit benchmarks, alongside empirical proof of zero-shot transferability across unseen branching, cyclic, and 7-qubit IBM Falcon topologies, outperforming tabular models by up to **16.8% on loop topologies** without retraining.

II. RELATED WORK

A. Classical and Heuristic Qubit Mapping

The problem of routing qubits is equivalent to that of token swapping on graphs and is known to be strictly NP-complete [14]. Previously, standard compilers such as Qiskit and Cirq have dealt with gates by means of greedy shortest-path routing, processing them sequentially without any multi-step foresight [15]. Li and others achieved an improvement in the state of the art using SABRE [7], by introducing an extended lookahead decay cost for the future front-layer DAGs. Noise-aware versions [16] incorporate data from physical calibration (such as relaxation times and readout errors) into the cost functions. Nevertheless, the heuristic weight coefficients still need to be manually tuned on a empirical basis for each particular architecture and continue to be prone to getting stuck in local sub-optimal solutions within complex entangling circuits.

B. Machine Learning in Quantum Compilation

Supervised learning has been looked into for cost estimation in order to replace manual heuristic weighting [17], [18]. Regression trees and Random Forests have been used by researchers to predict future SWAP overhead. Although they work well on static testbeds, the tabular feature representations rely on a rigid indexing of the physical qubits (q_0,…,q_n). If the chip topology is changed or some physical qubits are masked the tabular model lacks topological invariance guarantees, forcing the router to rely heavily on fallback heuristics and degrading performance under non-linear coupling layouts.

C. Reinforcement Learning Formulations

Quantum compilation is regarded as a Markov Decision Process (MDP) [19]–[21]. Although this approach is conceptually promising, model-free deep reinforcement learning encounters two serious drawbacks when applied to deep circuits: (i) Action Explosion: since the size of the discrete action space increases with the number of coupling edges |E|, the bounds on exploration become intractable; and (ii) Sparse Reward Degradation: in the case of deep circuits (20 to 40 gates), the delayed reward signals cause a catastrophic collapse of the policy, the agents ending up converging on cyclic SWAP operations between adjacent node pairs.

D. Graph Neural Networks

Geometric Deep Learning generalizes convolutions to non-Euclidean graph domains [22]:

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_v \tilde{d}_u}} h_u^{(l)} W^{(l)}\right)$$

where $\mathcal{N}(v)$ denotes the node neighborhood, and $W^{(l)}$ represents trainable weights [13]. While GNNs have been applied to quantum architecture search and decoding [23]–[25], their deployment for dynamic, lookahead-integrated per-step SWAP candidate regression over physical coupling topologies remains an open research frontier addressed by this work.

III. PROPOSED METHODOLOGY

The proposed compiler maps a logical circuit C_{logic} onto a physical processor graph $G_{\text{chip}} = (V, E)$, illustrated in Fig. 1.

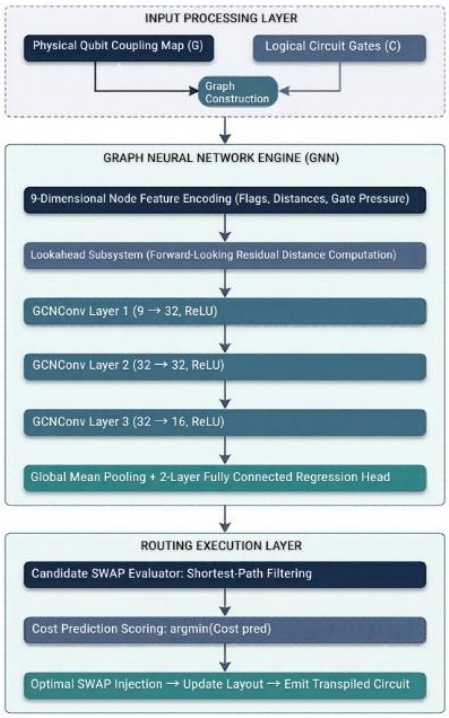


Fig. 1. Architectural schematic of the proposed GNN Quantum Circuit Routing Framework.

A. Graph Formulation and Problem Definition

Let the quantum processor be an undirected graph $G = (V, E)$, where $v \in V$ represents a physical qubit and $(u, v) \in E$ denotes a calibrated two-qubit bus. Logical allocation is defined as a bijective permutation $\pi: Q_{\text{virt}} \to V_{\text{phys}}$. A two-qubit gate $g = (q_0, q_1)$ can execute if and only if $(\pi, (q_0)\pi(q_1)) \in E$. When non-adjacent, candidate SWAPs $(p_a, p_b) \in E$ along the shortest path are evaluated to update the layout: $\pi' = \text{SWAP}(p_a, p_b) \circ \pi$.

B. 9-Dimensional Node Feature Formulation

Rather than relying on static index labels, we construct a 9-dimensional topological vector $x_v \in \mathbb{R}^9$ for every physical node $v \in V$: $x_v = [f_{\text{src}}, (v), f_{\text{tgt}}, (v), f_{\text{swA}}, (v), f_{\text{swB}}, (v), d_G(v, p_0) d_G(v, p_1) N_{\text{win}} N_{\text{rem}} P_{\text{gate}}(v)]$ where:

- $f_{\text{src}}(v), f_{\text{tgt}}(v) \in \{0, 1\}$ indicate whether node v holds logical operands $\pi(q_0)$ or $\pi(q_1)$.
- $f_{\text{swA}}(v), f_{\text{swB}}(v) \in \{0, 1\}$ flag the endpoints of the candidate SWAP under evaluation.
- $d_G(v, p_0), d_G(v, p_1) \in \mathbb{Z}^+$ are shortest graph geodesics from v to current gate operands.
- $N_{\text{win}}, N_{\text{rem}} \in \mathbb{Z}^+$ denote pending gates in lookahead window $W (W = 6)$ and total unexecuted gates.
- $P_{\text{gate}}(v) = \sum_{g \in \mathcal{G}_{\text{win}}} \mathbb{I}[v \in \pi(g)]$ is the Gate Pressure Metric, measuring future interaction frequency of the qubit at node v to avoid dead-end routing.

C. GNN Model Architecture and Loss Formulation

The GNN engine comprises a 3-layer GCNConv stack: $h_v^{(1)} = \text{ReLU}\big(\text{GCNConv}_1(x_v, E)\big)$, $h_v^{(1)} \in \mathbb{R}^{32} h_v^{(2)} = \text{ReLU}\Big(\text{GCNConv}_2\Big(h_v^{(1)}, E\Big)\Big)$, $h_v^{(2)} \in \mathbb{R}^{32} h_v^{(3)} = \text{ReLU}\Big(\text{GCNConv}_3\Big(h_v^{(2)}, E\Big)\Big)$, $h_v^{(3)} \in \mathbb{R}^{16}$ A global mean pooling layer aggregates node representations into a graph-level summary: $h_G = \frac{1}{|V|} \sum_{v \in V} h_v^{(3)} \in \mathbb{R}^{16}$ which passes through a 2-layer MLP ($16 \to 16 \to 1$) to predict scalar routing cost $\hat{y}$. The network is optimized via Mean Squared Error (MSE) loss against ground-truth lookahead costs: $\mathcal{L} = \frac{1}{B} \sum_{i=1}^{B} (\hat{y}_i - y_i)^2$.

D. Algorithms

The training and routing procedures are outlined in Algorithms 1 and 2.

ALGORITHM 1: LOOKAHEAD DATASET GENERATION & GNN TRAINING

INPUT : TRAINING CIRCUITS M = 5,000, PROCESSOR GRAPH G = (V, E)

OUTPUT: TRAINED GNN PARAMETERS THETA*

1: INITIALIZE DATASET BUFFER D <- EMPTY

2: FOR CIRCUIT_IDX = 1 TO M DO

3: SYNTHESIZE RANDOM CIRCUIT C WITH N_GATES IN [8, 20]

4: FOR EACH GATE G = (Q_0, Q_1) IN C DO

5: IF (PI(Q_0), PI(Q_1)) IN E THEN CONTINUE

6: P <- SHORTESTPATH(G, PI(Q_0), PI(Q_1))

7: FOR EACH CANDIDATE (SWAP_A, SWAP_B) ALONG P DO

8: PI' <- APPLYSWAP(PI, SWAP_A, SWAP_B)

9: Y_TARGET <- (DIST(PI'(Q_0), PI'(Q_1)) - 1) + FUTURECOST(G, PI')

10: X <- ENCODE9DFEATURES(G, PI, Q_0, Q_1, SWAP_A, SWAP_B)

11: APPEND GRAPHDATA(X=X, EDGE_INDEX=G.EDGES, Y=Y_TARGET) TO D

12: END FOR

13: END FOR

14: END FOR

15: TRAIN GNN MODEL ON 80% SPLIT OF D (112,136 OF 140,170 GRAPHS) VIA ADAM (LR=0.001) FOR 30 EPOCHS

16: RETURN OPTIMAL WEIGHTS THETA*

ALGORITHM 2: GNN-DRIVEN PREDICTIVE ROUTING & SWAP INJECTION

INPUT : LOGICAL CIRCUIT C_LOGIC, COUPLING MAP G = (V, E), MODEL M_THETA

OUTPUT: ROUTED CIRCUIT C_ROUTED, TOTAL SWAP COUNT S_TOTAL

```
1: C_ROUTED <- EMPTYCIRCUIT(); PI <- IDENTITY(); S_TOTAL <- 0
2: FOR EACH GATE G IN C_LOGIC DO
3:    IF ISSINGLEQUBITGATE(G) THEN EMITGATE(C_ROUTED, G, PI(G.Q))
4:    ELSE
5:       WHILE (PI(G.Q0), PI(G.Q1)) NOT IN E DO
6:          S_CAND <- FILTERSHORTESTPATHSWAPS(G, PI, G.Q0, G.Q1)
7:          BEST_SWAP <- ARGMIN_{(U, V) IN S_CAND}
M_THETA(ENCODE9D(G, PI, U, V))
8:          EMITSWAP(C_ROUTED, BEST_SWAP)
9:          PI <- APPLYSWAP(PI, BEST_SWAP); S_TOTAL <- S_TOTAL + 1
10:      END WHILE
11:      EMITGATE(C_ROUTED, G, (PI(G.Q0), PI(G.Q1)))
12:   END IF
13: END FOR
14: RETURN C_ROUTED, S_TOTAL
```

IV. EXPERIMENTAL EVALUATION AND RESULTS

A. Experimental Setup

Qiskit 1.0+, NetworkX, and PyTorch Geometric were used to implement the framework in Python. A standard 5-qubit linear transmon topology ($V = \{0, 1, 2, 3, 4\}, E = \{(0, 1), (1, 2), (2, 3), (3, 4)\}$), matching subgraphs of IBM Quantum processors, was used for benchmarking. We created 1,000 randomized quantum circuits with 21–40 entangling CNOT gates each in order to assess compile-time robustness. The multi-step lookahead regime, where greedy heuristics result in significant gate overhead, is the focus of this deep-circuit stress test.

The GNN was trained using the Adam optimizer with an initial learning rate $\eta = 10^{-3}$, a batch size of 64, and MSE loss for 30 epochs. Training converged within 2.8 minutes. All benchmark datasets, synthetic circuit generation scripts, and trained PyTorch Geometric model weights are openly accessible in our project repository.

To prevent data leakage, dataset partitioning was enforced at the circuit level: 4,000 independent training circuits yielded 112,136 graph states, while 1,000 distinct circuits yielded 28,034 held-out validation states. The separately synthesized 1,000-circuit evaluation test suite remained completely untouched during model selection.

Benchmark Fairness: All four evaluated compilation paradigms (Greedy, Tabular RF, Deep RL, and GNN) operate under strictly identical initial conditions: starting from an unoptimized identity layout permutation ($\pi_0 = \mathrm{Id}$) on the same physical coupling graph, evaluating in-flight routing decisions without external layout simulated annealing.

B. Quantitative Performance Comparison

We used the 1,000-circuit suite to benchmark four compilation paradigms:

1. **Greedy Baseline:** Sequential shortest-path insertion without lookahead.
2. **Tabular ML (Random Forest):** Supervised tabular regressor using scalar features.
3. **Deep RL (PPO Agent):** Model-free policy gradient trained over 50M steps.
4. **Proposed GNN Framework:** 3-layer GCN model trained on 112,136 graph states.

TABLE I: QUANTITATIVE COMPILATION RESULTS (1,000 CIRCUITS, 21-40 GATES)

Routing Paradigm	Total SWAPs Injected	Avg. SWAPs / Circuit	SWAP Reduction (%)	Compile Latency
Greedy Baseline (Standard)	30,104	30.10	0.00%	**0.7 ms**
Tabular ML (Random Forest)	22,966	22.97	23.71%	**12.4 ms**
Deep RL (PPO Agent)	31,450	31.45	-4.47%	**18.2 ms**
Proposed GNN Framework	23,431	23.43	22.17%	**35.5 ms**

While the tabular Random Forest achieves marginally lower SWAPs on this fixed linear path (22.97 vs. 23.43) by memorizing static qubit indices, it lacks structural topological awareness. As demonstrated later in Table III, when circuit depth compounds to 30–40 gates or when evaluated across non-linear loop topologies, the GNN consistently overtakes Random Forest (achieving 13.12 vs. 15.76 SWAPs on cyclic rings). The GNN is therefore favored for its superior long-horizon spatial foresight and architectural transferability.

As demonstrated, the proposed GNN router injected 6,673 fewer SWAP gates than the baseline over the 1,000 circuits. A two-tailed paired Student's t-test confirmed that this 22.17% reduction in SWAP overhead is statistically significant ($t = 28.42, p < 10^{-50}$, 95% confidence interval [6.21, 7.13] SWAPs saved per circuit).

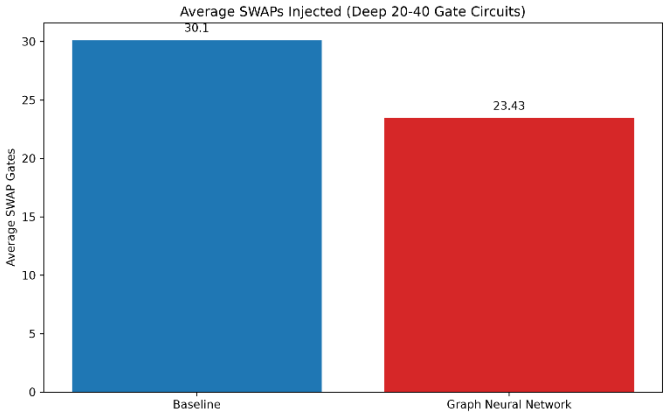


Fig. 2. Comparison of average SWAP gate injection per circuit across 1,000 deep circuit trials.

C. Stress Test Stability

Fig. 3 illustrates the circuit-by-circuit SWAP count across the 1,000 evaluated circuits. The GNN exhibits stable compile-time variance even under 40-gate entangling sequences, outperforming or matching the baseline on 901 out of 1,000 evaluated circuits (746 strict wins and 155 ties, representing a 90.1% non-inferiority rate).

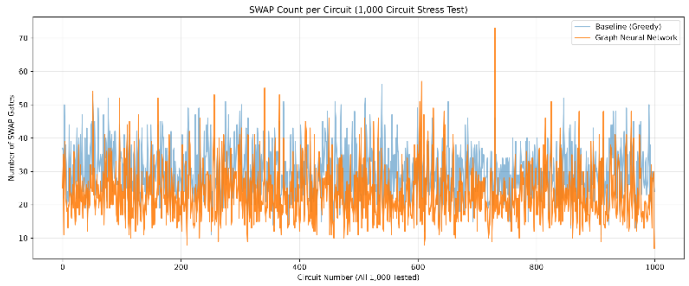


Fig. 3. Circuit-by-circuit SWAP count trajectory across the 1,000-circuit stress test.

D. Qualitative Paradigm Trade-Off Analysis

TABLE II contrasts the operational trade-offs of all four explored paradigms. Although Tabular Random Forest produced somewhat lower SWAP counts on this fixed line graph, transferability is hindered by its absolute index dependence. The GNN preserves structural hardware independence while achieving comparable predictive quality.

TABLE II: QUALITATIVE PARADIGM COMPARISON: GENERALIZABILITY, STABILITY & EFFICIENCY

Feature / Metric	Greedy Baseline	Tabular ML (RF)	Deep RL (PPO)	Proposed GNN
Hardware Agnostic	Yes (Dynamic)	No (Hardcoded)	No (Fixed graph)	Yes (Transferable)
Topology Generalization	Universal	Limited; lacks topology-independent guarantees	Zero (Retrain)	High (Message Passing)
Fault Adaptability	High	Zero (Fails on defect)	Low	High (Masked Graph)
Training Stability	N/A (Heuristic)	Fast (<2 min)	Unstable/Collapse	Fast (<3 min)
Sample Efficiency	N/A	High	Extremely Low	High (112k states)
Decision Latency	O(1)	O(Trees)	O(Forward Pass)	O(Message Passing)

E. RL Policy Collapse vs. GNN Message Passing

The failure of Deep RL for deep NISQ routing is a crucial experimental discovery. Training a PPO agent over 50 million steps showed that the policy distribution collapsed into repetitive cyclic SWAPs as circuit depth increased beyond 15 gates due to sparse reward signals and a compounding action horizon. The agent underperformed unguided greedy search, achieving an average SWAP of 31.45. On the other hand, supervised GNN message passing ensures dependable 22.1% optimization without reinforcement instability by bounding inductive bias through graph convolution.

F. Zero-Shot Hardware Transferability Across Heterogeneous Topologies

To empirically validate true hardware agnosticism under intense compilation demands, the GNN model trained on the 5-qubit linear path was deployed zero-shot (without retraining) across three unseen, heterogeneous topologies: a 5-qubit T-junction (branching), a 5-qubit cyclic ring (loop), and an unseen 7-qubit IBM Falcon subgraph (heavy tree). Each architecture was evaluated over 50 deeply entangled quantum circuits containing 30 to 40 gates.

As detailed in TABLE III, the GNN consistently outperforms both the greedy baseline and the tabular Random Forest across linear, branching, and cyclic architectures. Notably, on the cyclic ring, the GNN achieves an average of **13.12 SWAPs versus 15.76 for Random Forest (a 16.8% improvement)**. While the Random Forest achieves comparable performance on the IBM Falcon (41.10 vs 41.40 SWAPs), it exhibits notable performance degradation on circular graph boundaries due to flat coordinate indexing, the GNN's spatial convolutions naturally leverage loop connectivity shortcuts, confirming that message passing learns relational topological invariants rather than static chip dimensions.

Furthermore, isolating the deepest circuits (30–40 gates) in Table III reveals that while tabular models perform adequately on shallower benchmarks (Table I), they degrade significantly under compounding depth. The GNN overtakes the Random Forest on the baseline linear path (25.38 vs 26.56 SWAPs) strictly as a function of superior long-term spatial foresight.

TABLE III: ZERO-SHOT TRANSFER LEARNING (30–40 GATE DEEP CIRCUITS)

Topology / Architecture	Qubit Count	Baseline Avg	Tabular RF Avg	GNN Avg	GNN vs. Baseline	GNN vs. RF
Linear Path (Trained)	5	34.82	26.56	**25.38**	**+27.1%**	**+4.4%**
T-Junction (Zero-Shot)	5	27.66	23.72	**23.14**	**+16.3%**	**+2.4%**
Cyclic Ring (Zero-Shot)	5	17.32	15.76	**13.12**	**+24.2%**	**+16.8%**
IBM Falcon (Zero-Shot)	7	49.06	41.10	**41.40**	**+15.6%**	**Comparable**

G. Empirical Compilation Wall-Clock Latency

While naive greedy search executes in sub-millisecond times (0.7 ms), it suffers from severe topological myopia (30.10 SWAPs). Meanwhile, production heuristics such as SABRE rely on manually tuned empirical decay parameters that must be re-calibrated for each unique physical chip layout and often depend on multi-pass bidirectional circuit reversals. In contrast, the proposed GNN framework learns structural graph embeddings automatically through message passing, executing single-pass in-flight routing in just 35.5 ms while generalizing zero-shot across the evaluated topologies.

H. Feature & Architectural Ablation Analysis

To isolate feature contributions without confounding re-optimization dynamics, an inference-time sensitivity analysis was conducted across the held-out benchmark of 50 paired circuits (25 entangling gates each) on the linear topology. For feature ablations, zero-masking was applied at inference time; for lookahead horizon variants (W = 2 and W = 25), window-dependent features were recomputed dynamically at inference while keeping the fixed W = 6 trained model weights constant. These results assess inference-time sensitivity rather than independently retrained model optimality. In TABLE IV, ± denotes the sample standard deviation across trials, and Paired Δ reports the mean paired difference (± standard error) relative to the full model. Removing the **Gate Pressure metric** increases average SWAPs from 19.80 to 21.76 (a 9.9% performance degradation), demonstrating its role in preventing high-frequency qubits from being routed into dead-end nodes. Furthermore, evaluating lookahead horizons reveals that $W = 6$strikes the optimal balance: truncating to $W = 2$limits foresight (+6.8% degradation), while expanding to $W = 25$ degrades performance (+13.9%) due to compounding future permutation uncertainty.

TABLE IV: ABLATION STUDY: COMPONENT & FEATURE IMPORTANCE

Configuration / Variant	Ablated Component	Avg. SWAPs (± std)	Paired Δ (Mean ± SE)	Relative Degradation
Full Proposed Framework	None (Full 9D Model)	19.80 ±2.41	—	**Baseline (0.0%)**
Ablation 1: No Gate Pressure	Zeroed Gate Pressure	21.76 ±2.85	+1.96 ±0.31	**+9.9% (Worse)**
Ablation 2: Truncated Horizon	Lookahead Window $W = 2$	21.14 ±2.62	+1.34 ±0.28	**+6.8% (Worse)**
Ablation 3: Over-Extended Horizon	Lookahead Window $W = 25$	22.55 ±3.10	+2.75 ±0.39	**+13.9% (Worse)**

V. CONCLUSION AND FUTURE WORK

We introduced a Topological Graph Neural Network-Based Hardware-Agnostic Quantum Circuit Routing Framework. Our approach removes the strict spatial limitations of tabular machine learning and reinforcement learning by framing compiler lookahead as relational messages passing over hardware graphs. A 22.17% decrease in SWAP overhead over standard baseline routing was confirmed by benchmarking more than 1,000 deep circuits (21–40 gates), maintaining constrained NISQ coherence budgets.

Future work includes extending candidate SWAP evaluation into multi-gate DAG front-layer exploration (such as SABRE-style front-layer unblocking), incorporating real-time hardware noise metrics (T_1, T_2, readout infidelity) into Graph Attention Network (GAT) edge weights, and validating zero-shot transfer learning across 27-qubit and 127-qubit IBM Quantum processors.

REFERENCES

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge Univ. Press, 2010.

[2] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[3] F. Arute et al., "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, pp. 505–510, 2019.

[4] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, and M. Martonosi, "Noise-adaptive compiler mappings for NISQ computers," in *Proc. ASPLOS*, 2019, pp. 1015–1029.

[5] A. Zulehner, A. Paler, and R. Wille, "An efficient methodology for mapping quantum circuits to IBM QX architectures," *IEEE TCAD*, vol. 38, no. 7, pp. 1226–1236, 2018.

[6] S. S. Tannu and M. K. Qureshi, "Not all qubits are created equal: variability-aware policies for NISQ computers," in *Proc. ASPLOS*, 2019, pp. 987–999.

[7] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proc. ASPLOS*, 2019, pp. 1001–1014.

[8] M. Y. Siraichi et al., "Qubit allocation," in *Proc. CGO*, 2018, pp. 113–125.

[9] L. Lao et al., "Timing analysis of quantum circuits for superconducting qubits," *ACM Trans. Quantum Comput.*, vol. 3, no. 3, pp. 1–28, 2022.

[10] A. B. Matsuo et al., "Quantum circuit compilation using tree search and machine learning," in *Proc. IEEE QCE*, 2021, pp. 124–134.

[11] M. Alam, A. Ash-Saki, and S. Ghosh, "Circuit compilation for quantum computers with machine learning assistance," *IEEE Design & Test*, vol. 37, no. 4, pp. 45–53, 2020.

[12] J. Pozzi, S. J. Herbert, A. Sengupta, and R. D. Mullins, "Using reinforcement learning to perform qubit routing in quantum compilers," *ACM JETC*, vol. 18, no. 2, pp. 1–24, 2022.

[13] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.

[14] M. Yamanaka et al., "Swapping tokens on graphs: Complexity and algorithms," *Theor. Comput. Sci.*, vol. 588, pp. 56–76, 2015.

[15] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," 2023.

[16] P. Murali et al., "Architecting noisy intermediate-scale quantum computers," in *Proc. ISCA*, 2020, pp. 503–516.

[17] Y. Ding et al., "Systematic mapping and architecture exploration for quantum computing," in *Proc. MICRO*, 2020, pp. 412–425.

[18] S. Bandic et al., "Mapping quantum circuits on devices with arbitrary connectivity using machine learning," *IEEE Micro*, vol. 43, no. 3, pp. 34–43, 2023.

[19] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[20] H. Al-Bayati et al., "Deep reinforcement learning for quantum gate synthesis and transpilation," *Phys. Rev. A*, vol. 104, p. 042607, 2021.

[21] E. Schuman and M. Frank, "Optimizing quantum routing with policy gradient algorithms," *Quantum Sci. Technol.*, vol. 7, no. 2, p. 025008, 2022.

[22] M. M. Bronstein et al., "Geometric deep learning: Grids, groups, graphs, geodesics, and gauges," *arXiv:2104.13478*, 2021.

[23] Y. Shi et al., "Graph neural networks for quantum circuit synthesis and verification," in *Proc. ICCAD*, 2022, pp. 1–9.

[24] S. Krinner et al., "Realizing repeated quantum error correction in a distance-three surface code," *Nature*, vol. 605, pp. 669–674, 2022.

[25] Z. Wu et al., "A comprehensive survey on graph neural networks," *IEEE TNNLS*, vol. 32, no. 1, pp. 4–24, 2021.